CENTRAL BOARD OF SECONDARY EDUCATION

JAWAHAR NAVODAYA VIDYALAYA

TARIKHET ALMORA UTTARAKHAND 263663

COMPUTER SCIENCE (083)

PROJECT FILE

On

SCHOOL RESULT ANALYSER USING CSV FILE AND MODULES

SESSION: 2026-27

SUBMITTED BY :

S. No	Name	CBSE Roll
1.		
2.		
3.		
4.		

CLASS : XII
SECTION : SCIENCE

INTERNAL EXAMINER (PANKAJ BHILINGWAL)	EXTERNAL EXAMINER	PRINCIPAL (PANKAJ KUMAR SRIVASTAVA)

ACKNOWLEDGEMENT

I wish to express my deep sense of gratitude and indebtedness to our learned teacher Pankaj Bhilingwal, PGT Computer Science, J.N.V Tarikhet Almora for his invaluable help, advice and guidance in the preparation of this project file.

I am also greatly indebted to our principal Pankaj Kumar Srivastava and school authorities for providing me with the facilities and requisite laboratory conditions for making this project file.

I also extend my thanks to a number of teachers, my classmates and friends who helped me to complete this project file successfully.

Signature of Student

CERTIFICATE

This is to certify that,

S. No	Name	CBSE Roll

Student of Class XII, J.N.V Tarikhet Almora has completed the PROJECT FILE titled “School Result Analyser Using CSV File and Modules” during the academic year 2026-27 towards partial fulfilment of credit for the Computer Science Project evaluation of CBSE and submitted satisfactory report, as compiled in the following pages, under my supervision.

Internal Examiner

Signature

External Examiner

Signature

PREFACE

In the field of Computer Science, the term PROJECT refers to a computerization work that is completely performed by using a computer. Today almost every field is computerized, and the different tasks of every field are done using computers and appropriate computer programs. Such programs and software are developed in the form of Projects.

Learning Python in schools is a part of our syllabus, and according to the syllabus worked out by the CBSE, to which my school is affiliated, I enjoyed doing project work for the session 2026-27. I learned a lot, especially learning by doing was a great experience. The report is a summary of what I observed and learnt in school. My humble thanks go to all those concerned with my project.

I also feel that it is the right time to thank my teacher, who guided me during my project, and my parents, friends and project partners for the constant assistance and cooperation extended by them from time to time.

INDEX

S. No	Topic	Page No
1	About Project	6
2	Syllabus Concepts Used	8
3	Flow Chart	9
4	Project Code	10
5	Output	15
6	Technological Requirements: Hardware & Software	20
7	References	20

About Project

School Result Analyser Using CSV File and Modules

The School Result Analyser is a menu-driven Python program that reads the marks of Class XII students from a CSV file and prepares their result. It calculates total, percentage, grade and pass/fail status, finds subject-wise toppers, analyses the performance of the whole class and generates a report card for every student as a separate text file.

Unlike a single-file program, this project is divided into three files: main.py (menu), fileops.py (file work) and analysis.py (calculations). fileops and analysis are our own user-defined modules which are imported in main.py. This makes the program easier to read, test and reuse.

Project Overview

Objective:

The objective of this project is to learn how to create and use user-defined modules, and how to work with two types of files - CSV and text files - in one project. It also saves the time of a class teacher who has to calculate the result of many students.

Key Features:

1. **Display Result:** Shows marks, total, percentage, grade and result of all students in a table.
2. **Add Student Marks:** Adds a new student's marks to marks.csv after checking that marks are between 0 and 100.
3. **Search Student:** Shows the complete result of one student by roll number.
4. **Subject-wise Toppers:** Finds the highest scorer in every subject.
5. **Class Analysis:** Shows pass/fail count, pass percentage, class average, class topper and grade distribution.
6. **Generate Report Cards:** Creates a text file report card for every student in the reports folder.
7. **View Report Card:** Reads a report card text file and displays it.

Technology Stack

Programming Language: Python 3

Modules Used: csv, os, and user-defined modules fileops and analysis

fileops.py uses the csv module (csv.reader, csv.writer) to read and append marks, and text file functions (writelines, readline, read, tell, seek) for report cards. analysis.py contains only calculation functions with default arguments and multiple return values. main.py imports both modules and contains the menu. Grades follow the CBSE pattern (A1, A2, B1, B2, C1, C2, D, E) and a student must score at least 33 in every subject to pass.

Files in this Project

File	Purpose
main.py	Main program with menu - run this file
fileops.py	User-defined module for CSV and text file operations
analysis.py	User-defined module for calculations
marks.csv	CSV file with roll number, name and marks of 5 subjects

reports/	Folder of report card text files (created by option 6)
flowchart.py / flowchart.png	Flowchart program and image
img/	Screenshots of the output

How to Run the Program

1. Make sure Python 3 is installed on the computer.
2. Keep main.py, fileops.py, analysis.py and marks.csv in the SAME folder.
3. Open main.py in IDLE and press F5 (or run: python main.py).
4. marks.csv can also be opened and edited in MS-Excel.

Syllabus Concepts Used

This project covers the following topics of the CBSE Class XII Computer Science (083) syllabus:

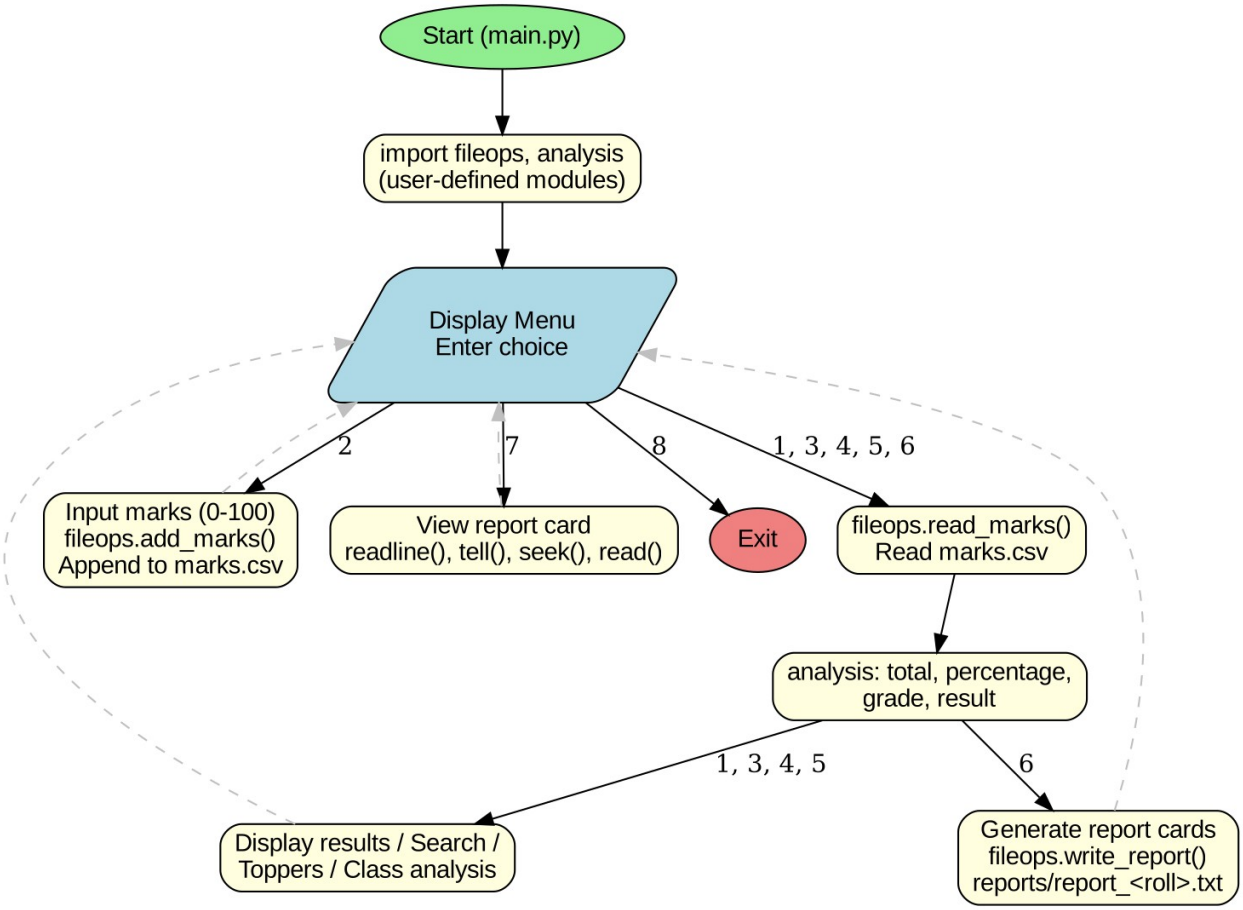
Concept	Where it is used in the project
User-defined modules	fileops.py and analysis.py imported with import and from ... import
CSV file handling	csv.reader() with next() to skip heading, csv.writer() with writerow()
Text file handling	writelines(), readline(), read(), tell() and seek() in fileops.py
Default arguments	percentage(total, subjects=5, max_marks=100), result(marks, pass_marks=33)
Returning multiple values	student_summary() returns total, percentage, grade and result as a tuple
Exception handling	try-except ValueError while entering marks
Lists and dictionaries	Student records as lists; grade counts in a dictionary
os module	os.path.exists() and os.mkdir() to create the reports folder

What is a module?

A module is a Python file (.py) containing functions and variables that can be used in other programs with the import statement. Python has built-in modules like csv, os and math, and we can also create our own modules.

Dividing a big program into modules keeps related functions together. In this project, all file related functions are in fileops.py and all calculations are in analysis.py, so the main program stays short and simple.

Flow Chart


Start (main.py)
import fileops, analysis
(user-defined modules)
Display Menu
Enter choice
2
7
8
1, 3, 4, 5, 6
Input marks (0-100)
fileops.add_marks()
Append to marks.csv
View report card
readline(), tell(), seek(), read()
Exit
fileops.read_marks()
Read marks.csv
analysis: total, percentage,
grade, result
1, 3, 4, 5
6
Display results / Search /
Toppers / Class analysis
Generate report cards
fileops.write_report()
reports/report_<roll>.txt

Project Code

main.py

```
# main.py
# School Result Analyser - menu driven program
# Run this file. It uses our own modules fileops.py and analysis.py

import fileops
import analysis
from fileops import SUBJECTS


def get_marks(subject):
    while True:
        try:
            m = int(input("Enter marks in " + subject + " (0-100): "))
            if 0 <= m <= 100:
                return m
            print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter a number.")


def display_all():
    students = fileops.read_marks()
    if len(students) == 0:
        print("No records found.")
        return
    print("%-5s %-16s %5s %5s %5s %5s %5s %6s %7s %5s %6s" %
          ("Roll", "Name", "Eng", "Phy", "Chem", "Math", "CS", "Total", "Per%",
"Grade", "Result"))
    print("-" * 82)
    for s in students:
        t, p, g, r = analysis.student_summary(s)
        print("%-5s %-16s %5d %5d %5d %5d %5d %6d %7.2f %5s %6s" %
              (s[0], s[1], s[2], s[3], s[4], s[5], s[6], t, p, g, r))


def add_student():
    students = fileops.read_marks()
    roll = input("Enter Roll Number: ")
    for s in students:
        if s[0] == roll:
            print("Roll number already exists!")
            return
    name = input("Enter Student Name: ")
    row = [roll, name]
    for sub in SUBJECTS:
        row.append(get_marks(sub))
    fileops.add_marks(row)
    print("Marks of", name, "saved successfully.")


def search_student():
    roll = input("Enter Roll Number to search: ")
    for s in fileops.read_marks():
        if s[0] == roll:
            t, p, g, r = analysis.student_summary(s)
            print("Name:", s[1])
            for i in range(len(SUBJECTS)):
                print("  ", SUBJECTS[i], ":", s[i + 2])
            print("Total:", t, "| Percentage:", p, "| Grade:", g, "| Result:", r)
            return
    print("No student with roll number", roll)


def subject_toppers():
    students = fileops.read_marks()
```

```
    if len(students) == 0:
        print("No records found.")
        return
    print("Subject-wise Toppers")
    print("--------------------")
    for i in range(len(SUBJECTS)):
        name, marks = analysis.subject_topper(students, i + 2)
        print("%-10s : %-16s (%d)" % (SUBJECTS[i], name, marks))


def class_analysis():
    students = fileops.read_marks()
    if len(students) == 0:
        print("No records found.")
        return
    passed = 0
    grades = {}
    topper = ""
    top_per = -1
    for s in students:
        t, p, g, r = analysis.student_summary(s)
        if r == "PASS":
            passed = passed + 1
        grades[g] = grades.get(g, 0) + 1
        if p > top_per:
            top_per = p
            topper = s[1]
    print("Total Students :", len(students))
    print("Passed         :", passed)
    print("Failed         :", len(students) - passed)
    print("Pass Percentage:", round(passed * 100 / len(students), 2), "%")
    print("Class Average  :", analysis.class_average(students), "%")
    print("Class Topper   :", topper, "(", top_per, "% )")
    print("Grade Distribution:")
    for g in ["A1", "A2", "B1", "B2", "C1", "C2", "D", "E"]:
        if g in grades:
            print("  ", g, ":", "*" * grades[g], grades[g])


def generate_reports():
    students = fileops.read_marks()
    count = 0
    for s in students:
        t, p, g, r = analysis.student_summary(s)
        lines = []
        lines.append("REPORT CARD - CLASS XII\n")
        lines.append("Roll No : " + s[0] + "\n")
        lines.append("Name    : " + s[1] + "\n")
        lines.append("-" * 28 + "\n")
        for i in range(len(SUBJECTS)):
            lines.append("%-12s %5d\n" % (SUBJECTS[i], s[i + 2]))
        lines.append("-" * 28 + "\n")
        lines.append("Total      : " + str(t) + " / 500\n")
        lines.append("Percentage : " + str(p) + " %\n")
        lines.append("Grade      : " + g + "\n")
        lines.append("Result     : " + r + "\n")
        fileops.write_report(s[0], lines)
        count = count + 1
    print(count, "report cards created in the folder '" + fileops.REPORT_FOLDER +
"'")


def view_report():
    roll = input("Enter Roll Number: ")
    content, position = fileops.read_report(roll)
    if content is None:
        print("Report not found. Generate report cards first (option 6).")
        return
    print("(File pointer after reading first line was at byte", position, ")")
    print(content)
```

```
def main():
    while True:
        print("\n====== SCHOOL RESULT ANALYSER ======")
        print("1. Display Result of All Students")
        print("2. Add Student Marks")
        print("3. Search Student")
        print("4. Subject-wise Toppers")
        print("5. Class Analysis")
        print("6. Generate Report Cards (text files)")
        print("7. View a Report Card")
        print("8. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            display_all()
        elif choice == "2":
            add_student()
        elif choice == "3":
            search_student()
        elif choice == "4":
            subject_toppers()
        elif choice == "5":
            class_analysis()
        elif choice == "6":
            generate_reports()
        elif choice == "7":
            view_report()
        elif choice == "8":
            print("Goodbye!")
            break
        else:
            print("Invalid choice! Please try again.")


main()
```

fileops.py

```
# fileops.py
# User-defined module: all FILE related work (CSV file and text files)

import csv
import os

MARKS_FILE = "marks.csv"
REPORT_FOLDER = "reports"
SUBJECTS = ["English", "Physics", "Chemistry", "Maths", "CS"]


def read_marks():
    # Returns a list of students. Each student is a list:
    # [roll, name, m1, m2, m3, m4, m5]  (marks are converted to int)
    students = []
    if not os.path.exists(MARKS_FILE):
        return students
    f = open(MARKS_FILE, "r", newline="")
    reader = csv.reader(f)
    next(reader)                        # skip the heading row
    for row in reader:
        student = [row[0], row[1]]
        for m in row[2:]:
            student.append(int(m))
        students.append(student)
    f.close()
    return students


def add_marks(row):
    # Append one student row to the CSV file
```

```
    new_file = not os.path.exists(MARKS_FILE)
    f = open(MARKS_FILE, "a", newline="")
    writer = csv.writer(f)
    if new_file:
        writer.writerow(["RollNo", "Name"] + SUBJECTS)
    writer.writerow(row)
    f.close()


def write_report(roll, lines):
    # Write a report card (list of lines) into a text file
    if not os.path.exists(REPORT_FOLDER):
        os.mkdir(REPORT_FOLDER)
    file_name = REPORT_FOLDER + "/report_" + roll + ".txt"
    f = open(file_name, "w")
    f.writelines(lines)
    f.close()
    return file_name


def read_report(roll):
    # Read a report card text file and show tell() / seek() use
    file_name = REPORT_FOLDER + "/report_" + roll + ".txt"
    if not os.path.exists(file_name):
        return None, 0
    f = open(file_name, "r")
    first_line = f.readline()          # read only the first line
    position = f.tell()                # where the file pointer is now
    f.seek(0)                          # go back to the beginning
    content = f.read()                 # read the whole file
    f.close()
    return content, position
```

analysis.py

```
# analysis.py
# User-defined module: all CALCULATIONS on marks


def total(marks):
    t = 0
    for m in marks:
        t = t + m
    return t


def percentage(total_marks, subjects=5, max_marks=100):
    # Default arguments: 5 subjects of 100 marks each
    return round(total_marks * 100 / (subjects * max_marks), 2)


def grade(per):
    # CBSE style grades
    if per >= 91:
        return "A1"
    elif per >= 81:
        return "A2"
    elif per >= 71:
        return "B1"
    elif per >= 61:
        return "B2"
    elif per >= 51:
        return "C1"
    elif per >= 41:
        return "C2"
    elif per >= 33:
        return "D"
    else:
        return "E"
```

```
def result(marks, pass_marks=33):
    # A student passes only if every subject has at least 33 marks
    for m in marks:
        if m < pass_marks:
            return "FAIL"
    return "PASS"


def student_summary(student):
    # Returns MORE THAN ONE value (a tuple)
    marks = student[2:]
    t = total(marks)
    p = percentage(t)
    return t, p, grade(p), result(marks)


def subject_topper(students, index):
    # index = position of the subject in the student list (2 to 6)
    best = students[0]
    for s in students:
        if s[index] > best[index]:
            best = s
    return best[1], best[index]


def class_average(students):
    if len(students) == 0:
        return 0
    sum_per = 0
    for s in students:
        t, p, g, r = student_summary(s)
        sum_per = sum_per + p
    return round(sum_per / len(students), 2)
```

Output

Output 1: Result of All Students

Python Shell - Result of All Students

```
====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 1
Roll  Name              Eng   Phy  Chem  Math   CS  Total    Per% Grade Result
-------------------------------------------------------------------------------
1     Aarav Negi         88    76    81    92    95    432   86.40    A2   PASS
2     Bhawna Bisht       92    85    89    78    98    442   88.40    A2   PASS
3     Chetan Rawat       65    58    61    49    72    305   61.00    B2   PASS
4     Diya Joshi         79    91    84    95    90    439   87.80    A2   PASS
5     Gaurav Mehra       45    30    52    38    60    225   45.00    C2   FAIL
6     Kavya Pandey       71    66    70    63    81    351   70.20    B2   PASS
7     Mohit Bora         56    48    44    51    67    266   53.20    C1   PASS
8     Neha Adhikari      95    89    93    97    99    473   94.60    A1   PASS
```

Output 2: Adding Student Marks

Python Shell - Adding Student Marks

```
====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 2
Enter Roll Number: 9
Enter Student Name: Priya Kandpal
Enter marks in English (0-100): 84
Enter marks in Physics (0-100): ninety
Please enter a number.
Enter marks in Physics (0-100): 90
Enter marks in Chemistry (0-100): 77
Enter marks in Maths (0-100): 120
Marks must be between 0 and 100.
Enter marks in Maths (0-100): 69
Enter marks in CS (0-100): 88
Marks of Priya Kandpal saved successfully.
```

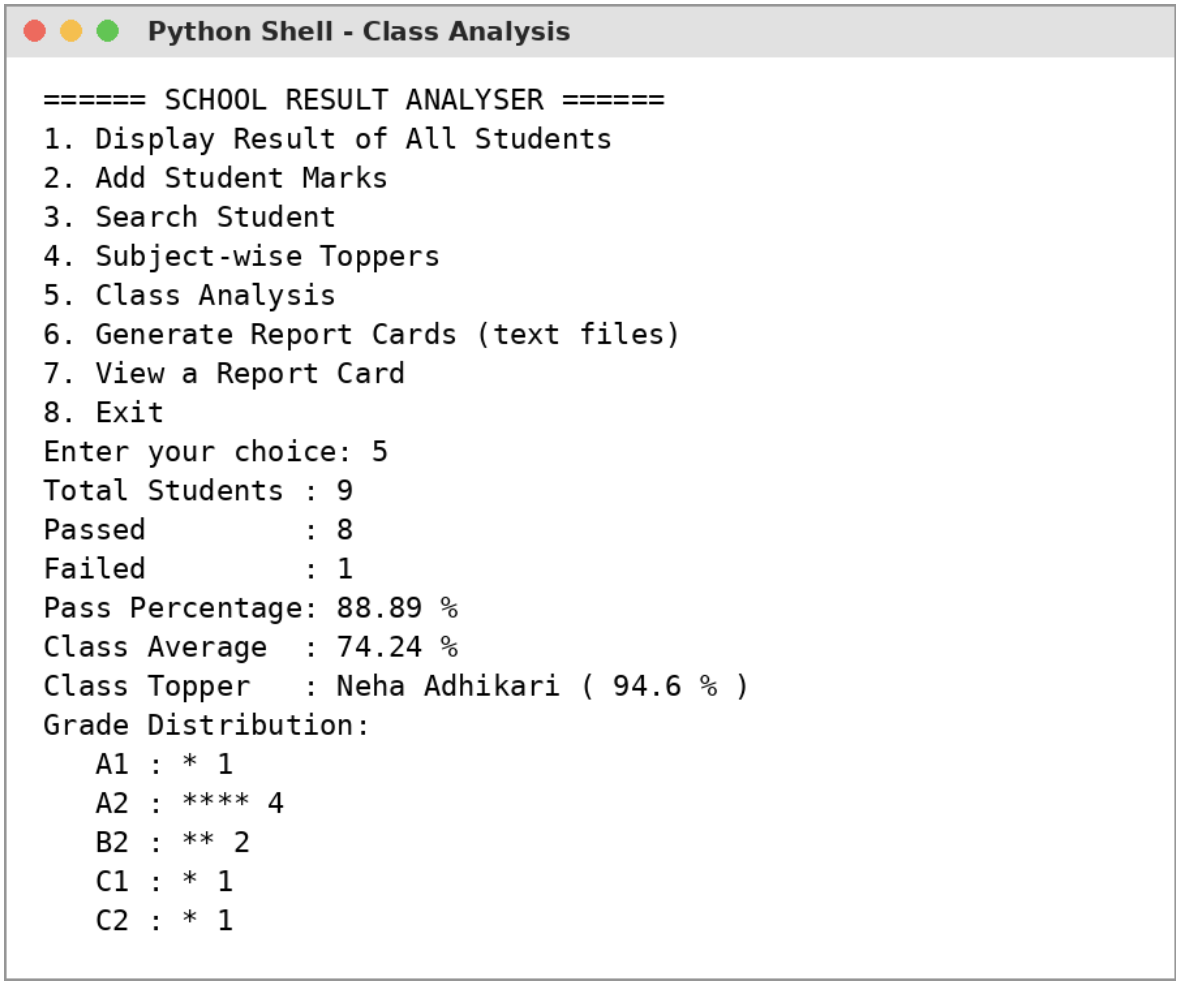
Output 3: Search and Subject Toppers

Python Shell - Search and Subject Toppers

```
====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 3
Enter Roll Number to search: 5
Name: Gaurav Mehra
   English : 45
   Physics : 30
   Chemistry : 52
   Maths : 38
   CS : 60
Total: 225 | Percentage: 45.0 | Grade: C2 | Result: FAIL

====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 4
Subject-wise Toppers
--------------------
English    : Neha Adhikari    (95)
Physics    : Diya Joshi       (91)
Chemistry  : Neha Adhikari    (93)
Maths      : Neha Adhikari    (97)
CS         : Neha Adhikari    (99)
```

Output 4: Class Analysis



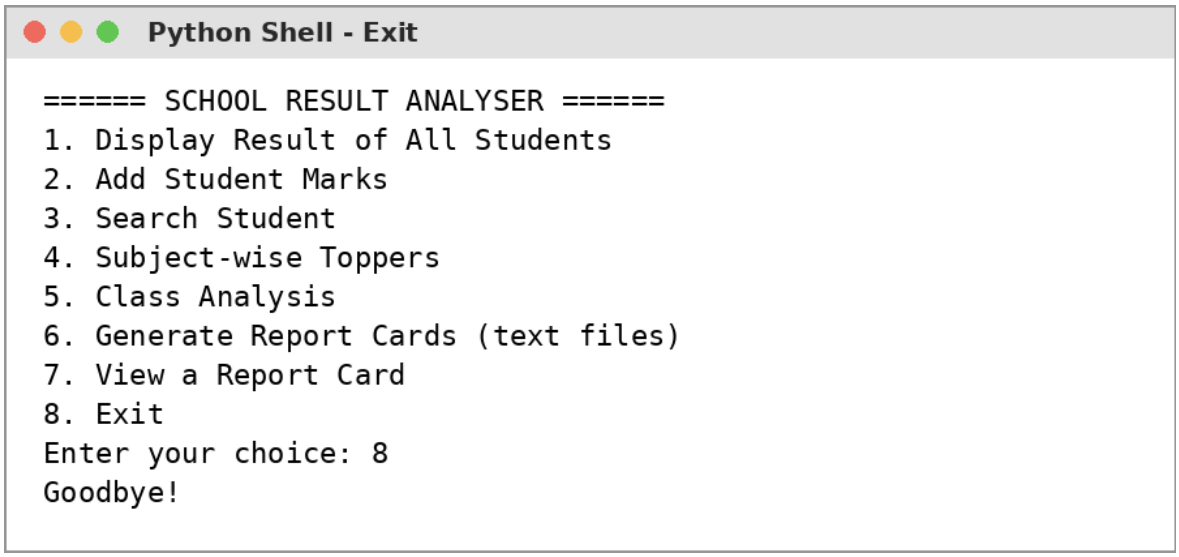
Output 5: Report Cards (Text Files)

Python Shell - Report Cards (Text Files)

```
====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 6
9 report cards created in the folder 'reports'

====== SCHOOL RESULT ANALYSER ======
1. Display Result of All Students
2. Add Student Marks
3. Search Student
4. Subject-wise Toppers
5. Class Analysis
6. Generate Report Cards (text files)
7. View a Report Card
8. Exit
Enter your choice: 7
Enter Roll Number: 8
(File pointer after reading first line was at byte 24 )
REPORT CARD - CLASS XII
Roll No : 8
Name    : Neha Adhikari
----------------------------
English         95
Physics         89
Chemistry       93
Maths           97
CS              99
----------------------------
Total      : 473 / 500
Percentage : 94.6 %
Grade      : A1
Result     : PASS
```

Output 6: Exit



marks.csv (data file)

```
RollNo,Name,English,Physics,Chemistry,Maths,CS
1,Aarav Negi,88,76,81,92,95
2,Bhawna Bisht,92,85,89,78,98
3,Chetan Rawat,65,58,61,49,72
4,Diya Joshi,79,91,84,95,90
5,Gaurav Mehra,45,30,52,38,60
6,Kavya Pandey,71,66,70,63,81
7,Mohit Bora,56,48,44,51,67
8,Neha Adhikari,95,89,93,97,99
9,Priya Kandpal,84,90,77,69,88
```

Technological Requirements: Hardware & Software

Software

- Python 3.x (IDLE)
- VS Code / any text editor
- Microsoft Excel (to view marks.csv)

Hardware

- Laptop / PC with Intel Core i3 or above
- 4 GB RAM or more
- Keyboard and Mouse

References

1. Computer Science with Python - Textbook for Class XII (Sumita Arora)
2. NCERT Computer Science Textbook for Class XII, Chapters: Functions, File Handling
3. https://docs.python.org/3/library/csv.html
4. https://docs.python.org/3/tutorial/modules.html
5. https://www.programiz.com/python-programming/csv